Amplicon Pipeline — User Guide

Overview

This pipeline processes paired-end Illumina amplicon sequencing data through three stages:

1. **cutadapt** — primer trimming and removal of reads without primers
2. **bbduk** — adapter trimming and quality filtering
3. **bbmerge** — merging of overlapping read pairs into single amplicon sequences

One PBS job is submitted per locus, so COI and ITS2 (for example) run simultaneously as separate jobs. All samples for a given locus are processed sequentially within that job.

Files

File	Purpose
`submit_amplicon_pipeline.sh`	Master submission script — reads `primers.txt` and submits one PBS job per locus
`amplicon_pipeline.pbs`	PBS job script — runs cutadapt → bbduk → bbmerge for one locus
`primers.txt`	Locus definitions (primer names and sequences)
`sequences.txt`	List of sample filenames from the sequencer

Both scripts must be in the same directory. Do not run `amplicon_pipeline.pbs` directly — always submit via `submit_amplicon_pipeline.sh`.

Input File Formats

primers.txt

One locus per line, whitespace-delimited. Lines beginning with `#` are ignored.

```
# locus_name   fwd_name      fwd_sequence              rev_name      rev_sequence
COI             mlCOIintF     GGWACWGGWTGAACWGTWTAYCCYCC  jgHCO2198   TAIACYTCIGGRTGICCRAARAAYCA
ITS2_Plant      ITS-S2F       ATGCGATACTTGGTGTGAAT          ITS4        TCCTCCGCTTATTGATATGC
```

Degenerate IUPAC bases (R, Y, W, S, K, M, B, D, H, V, N, I) are detected automatically and cutadapt is run with `--match-read-wildcards` for those loci.

sequences.txt

A plain text file with one filename per line — typically the R1 filenames from the sequencer, though any read filename will work. The pipeline strips the read suffix automatically, so either R1 or R2 filenames (or a mix) are fine.

```
Sample001_R1_001.fastq.gz
Sample002_R1_001.fastq.gz
Sample003_R1_001.fastq.gz
```

The pipeline supports all common Illumina naming conventions:

Pattern	Example
Illumina BCL2FASTQ default	`Sample001_R1_001.fastq.gz`
Simple R1/R2	`Sample001_R1.fastq.gz`
Numeric 1/2	`Sample001_1.fastq.gz`
fq.gz extension	`Sample001_R1.fq.gz`
BCL2FASTQ with fq.gz	`Sample001_R1_001.fq.gz`

The naming convention is auto-detected from the files present in the input directory — you do not need to specify it.

Directory Structure

Set up your project directory as follows before running:

```
/path/to/project/
├── primers.txt
├── sequences.txt
├── submit_amplicon_pipeline.sh
├── amplicon_pipeline.pbs
└── raw/                        ← raw reads go here (or specify with -i)
    ├── Sample001_R1_001.fastq.gz
    ├── Sample001_R2_001.fastq.gz
    ├── Sample002_R1_001.fastq.gz
    └── Sample002_R2_001.fastq.gz
```

Running the Pipeline

Basic usage (reads in `<base_path>/raw/`)

```
cd /path/to/project
bash submit_amplicon_pipeline.sh -b /path/to/project
```

Specifying a different raw reads directory

```
bash submit_amplicon_pipeline.sh \
    -b /path/to/project \
    -i /path/to/raw_reads
```

Full options

```
bash submit_amplicon_pipeline.sh \
    -b /path/to/project \        # output root directory
    -i /path/to/raw_reads \      # raw reads directory [default: <base_path>/raw]
    -s /path/to/sequences.txt \  # sample list         [default: ./sequences.txt]
    -p /path/to/primers.txt \    # primer definitions  [default: ./primers.txt]
    -w 24:00:00 \                # walltime            [default: 24:00:00]
    -m 32gb \                    # memory per job      [default: 32gb]
    -n 40                        # CPUs per job        [default: 40]
```

Monitor running jobs

```
qstat -u $USER
```

Outputs

All outputs are written under `<base_path>/`. One set of directories is created per locus.

```
<base_path>/
├── logs/
│   ├── amplicon_COI.log                ← main PBS job log per locus
│   ├── cutadapt_COI_Sample001.log      ← per-tool per-sample logs
│   ├── bbduk_COI_Sample001.log
│   └── bbmerge_COI_Sample001.log
│
├── commands/
│   ├── cutadapt_COI.cmds              ← exact commands run (for reproducibility)
│   ├── bbduk_COI.cmds
│   └── bbmerge_COI.cmds
│
├── COI_cutadapt/                       ← primer-trimmed reads
│   ├── Sample001_R1.fq.gz
│   └── Sample001_R2.fq.gz
│
├── COI_bbduk/                          ← adapter-trimmed, quality-filtered reads
│   ├── Sample001_R1.fq.gz
│   └── Sample001_R2.fq.gz
│
├── COI_merged/                         ← merged amplicon reads (DADA2 input)
│   ├── Sample001_merged.fq.gz
│   ├── Sample001_unmerged_R1.fq.gz
│   └── Sample001_unmerged_R2.fq.gz
│
├── COI_merge_hist/                     ← insert size histograms
│   └── Sample001.hist
│
├── cutadapt_COI_summary.tsv           ← summary table: reads in/out per sample
├── bbduk_COI_summary.tsv
└── bbmerge_COI_summary.tsv
```

Summary TSV format

Each stage produces a TSV summarising read counts:

cutadapt_\<locus>_summary.tsv

```
Sample          Input_Pairs  Pairs_Written  Pct_Written
Sample001       1500000      1234567        82.3
```

bbduk_\<locus>_summary.tsv

```
Sample          Input_Reads  Result_Reads  Pct_Remaining
Sample001       2469134      2401022       97.2
```

bbmerge_\<locus>_summary.tsv

```
Sample          Pairs    Joined   Pct_Joined
Sample001       1200511  1156823  96.4
```

Resource Tuning

Memory

The default `32gb` is sufficient for most amplicon datasets. If bbmerge exits with an out-of-memory error, increase to `64gb`:

```
bash submit_amplicon_pipeline.sh -b /path/to/project -m 64gb
```

CPUs

The default `40` CPUs matches the thread count used by cutadapt, bbduk and bbmerge internally. Adjust to match your HPC queue limits:

```
bash submit_amplicon_pipeline.sh -b /path/to/project -n 16
```

Note: if you change CPUs here you may also need to update `THREADS=40` inside `amplicon_pipeline.pbs` to match, or the tools will request more threads than the job was allocated.

Walltime

For a typical run of ~384 samples across two loci, `24:00:00` is usually sufficient. Very large datasets (>1000 samples) may need `48:00:00`.

Troubleshooting

Job fails immediately with "cutadapt not found"

The PBS script loads `module load python/3.11.4` which should include cutadapt. Check that it is installed in that module:

```
module load python/3.11.4
cutadapt --version
```

Read files not found

The submit script auto-detects the file naming convention by scanning the input directory. If it cannot detect a pattern it will exit with an error listing what it looked for. Check that your filenames match one of the supported patterns and that `-i` points to the correct directory.

bbmerge low merge rate

A merge rate below ~70% usually means the amplicon is too long for the read length, so the reads do not overlap sufficiently. Check that your expected amplicon length is less than `2 × read_length − 20 bp`. For ITS2 (~300–400 bp) with 250 bp reads this is at the limit — consider using the unmerged reads as well.

Samples with 0 reads after cutadapt

This means the primer sequences were not found in those reads. Check: - The forward and reverse primer sequences are correct and in 5'→3' orientation - The locus is actually present in those samples - If primers are degenerate, `WILDCARD=1` should appear in the job log

---

Post-pipeline: Plotting and QC (`plot_pipeline_summary.py`)

After all PBS jobs have finished, run `plot_pipeline_summary.py` to combine the per-locus TSV summaries and generate QC plots. This script works with output from both the PBS pipeline and the Python pipeline.

Requirements

```
pip install matplotlib numpy pandas
# or on the HPC:
module load python/3.11.4
```

Usage

```
# Minimal — primers.txt auto-detected from base_path
python3 plot_pipeline_summary.py -b /path/to/project

# With explicit primers.txt
python3 plot_pipeline_summary.py \
    -b /path/to/project \
    -p /path/to/primers.txt
```

TSV discovery

The script tries to find summary data in this order:

1. **Combined TSV** (written by the Python pipeline): `<base_path>/cutadapt_summary.tsv` etc.
2. **Per-locus TSVs** (written by the PBS pipeline): `<base_path>/cutadapt_<LOCUS>_summary.tsv` etc.

If per-locus files are used they are merged and written out as combined TSVs, so the script runs faster on any subsequent reruns.

Outputs

All plots are written to `<base_path>/`:

File	Contents
`pipeline_summary.png`	

File	Contents
	Overview of reads surviving each stage per sample per locus
`cutadapt_summary.png`	Reads written vs discarded by cutadapt per locus
`bbduk_bbmerge_summary.png`	bbduk retention and bbmerge merge rate per locus
`<locus>_merge_hist/combined_histogram.tsv`	Combined insert-size counts across all samples
`<locus>_merge_hist/combined_histogram.png`	Insert-size histogram with mean, median and mode

The insert-size histogram shows the distribution of merged amplicon lengths across all samples. The mean, median and mode are shown as dashed lines. A tight peak at the expected amplicon length (e.g. ~313 bp for COI minibarcode, ~300 bp for ITS2) indicates good primer specificity. A broad distribution or secondary peaks suggest off-target amplification or primer dimer.

Typical workflow

```
# 1. Submit pipeline jobs
bash submit_amplicon_pipeline.sh -b /path/to/project

# 2. Wait for jobs to finish
qstat -u $USER

# 3. Generate QC plots
module load python/3.11.4
python3 plot_pipeline_summary.py -b /path/to/project

# 4. Review outputs
ls /path/to/project/*.png
ls /path/to/project/*_merge_hist/combined_histogram.png
```

The merged reads in `<locus>_merged/` are ready for DADA2 or vsearch amplicon processing. Feed the `<base_path>` and locus name into the downstream pipeline:

```
# DADA2 pipeline
Rscript DADA_HPC.R \
    --mode   full \
    -b       /path/to/project \
    -l       COI \
```

```
    --threads 16

# vsearch pipeline
python3 vsearch_HPC.py \
    --mode   full \
    -b       /path/to/project \
    -l       COI \
    --threads 16
```